

ROYAUME DU MAROC

Office de la Formation Professionnelle et de la Promotion du Travail

Centre de Formation Al Kendi — Casablanca

Brevet de Technicien Supérieur (BTS)

Filière : Développement des Applications Informatiques



Projet de Fin d'Etudes

PulmoScan AI

*Application mobile de pre-diagnostic des pathologies pulmonaires
par intelligence artificielle embarquée*

Realise par :

Foudali Kenza

El Hajji Adnane

Encadre par :

Mme. Amina Aboulmira

Annee universitaire 2025/2026

Dédicace

*À nos chers parents,
qui n'ont jamais cessé de croire en nous,
et dont les sacrifices silencieux ont éclairé chaque étape de notre parcours.*

*À nos frères et sœurs,
sources inépuisables de soutien et d'encouragement.*

*À nos professeurs,
qui ont semé en nous le goût du savoir et la rigueur du travail.*

*À tous nos amis,
avec qui nous avons partagé les joies et les difficultés de ces années de formation.*

*À tous ceux qui, de près ou de loin,
ont contribué à faire de ce projet une réalité,
nous dédions ce modeste travail.*

Kenza & Adnane

Remerciements

Avant toute chose, nous tenons à exprimer notre profonde gratitude envers toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'études.

Nous adressons nos remerciements les plus sincères à notre encadrante, **Mme. Amina Aboulmira**, pour la qualité de son encadrement, sa disponibilité constante, ses conseils avisés et la confiance qu'elle nous a accordée tout au long de ce travail. Sa rigueur scientifique et son sens pédagogique ont été déterminants dans l'aboutissement de ce projet.

Nous remercions également l'ensemble du corps professoral et de l'équipe administrative du **Centre de Formation Al Kendi de Casablanca**, qui nous ont accompagnés durant ces deux années de formation au sein de la filière Développement des Applications Informatiques. La qualité de l'enseignement reçu a constitué le socle indispensable à la réussite de ce projet.

Nos remerciements vont aussi aux membres du jury, qui nous font l'honneur d'évaluer ce travail. Nous espérons que ce rapport saura répondre à leurs attentes et susciter leur intérêt.

Enfin, nous exprimons notre reconnaissance à nos familles et à nos amis, dont le soutien moral, la patience et les encouragements n'ont jamais fait défaut, particulièrement dans les moments les plus exigeants de ce parcours.

À toutes et à tous, nous disons : *merci*.

Résumé

PulmoScan AI est une application mobile multiplateforme, développée avec le framework Flutter, destinée à assister le personnel médical dans le pré-diagnostic des pathologies pulmonaires à partir de radiographies thoraciques. L'application embarque deux modèles d'intelligence artificielle exécutés localement grâce à TensorFlow Lite : un réseau de classification **DenseNet121** (issu de la bibliothèque TorchXRayVision et entraîné sur le jeu de données NIH ChestX-ray14, comprenant 112 120 radiographies annotées) capable de détecter 14 pathologies pulmonaires, et un réseau de segmentation **U-Net** qui isole les champs pulmonaires sur l'image.

La particularité fondamentale de PulmoScan AI réside dans son architecture *offline-first* : l'intégralité du traitement — acquisition de l'image, inférence des modèles, stockage des données et génération des rapports PDF — s'effectue sur l'appareil, sans aucune dépendance à un serveur distant. Les données des patients sont conservées dans une base de données locale SQLite, et l'authentification repose sur un hachage SHA-256 des mots de passe. Cette approche garantit la confidentialité des données médicales, leur disponibilité hors connexion et une conformité accrue en matière de protection des données personnelles.

Ce rapport présente le contexte du projet, la conception détaillée du système selon une démarche Scrum adaptée, l'implémentation technique du pipeline d'inférence, les interfaces réalisées et les campagnes de tests menées pour valider la fiabilité de la solution.

Mots-clés : Intelligence artificielle, radiographie thoracique, DenseNet121, U-Net, TensorFlow Lite, Flutter, SQLite, pré-diagnostic, offline-first, santé numérique.

Abstract

PulmoScan AI is a cross-platform mobile application, built with the Flutter framework, designed to assist medical staff in the pre-diagnosis of pulmonary pathologies from chest X-rays. The application embeds two artificial intelligence models running locally through TensorFlow Lite : a **DenseNet121** classification network (from the TorchXRayVision library, trained on the NIH ChestX-ray14 dataset of 112,120 annotated radiographs) capable of detecting 14 lung pathologies, and a **U-Net** segmentation network that isolates the lung fields in the image.

The core distinguishing feature of PulmoScan AI is its *offline-first* architecture : the entire workflow — image acquisition, model inference, data storage and PDF report generation — runs on-device, with no dependency on any remote server. Patient data is stored in a local SQLite database, and authentication relies on SHA-256 password hashing. This approach guarantees the confidentiality of medical data, its offline availability and improved compliance with personal data protection requirements.

This report presents the project context, the detailed system design following an adapted Scrum methodology, the technical implementation of the inference pipeline, the developed interfaces and the testing campaigns carried out to validate the reliability of the solution.

Keywords : Artificial intelligence, chest X-ray, DenseNet121, U-Net, TensorFlow Lite, Flutter, SQLite, pre-diagnosis, offline-first, digital health.

Résumé en arabe

(Cette page contient le résumé en langue arabe.)

Note technique

Le résumé en arabe nécessite une compilation avec **XeLaTeX** et un moteur de support bidirectionnel pour un affichage correct des caractères arabes. Afin de garantir la compilation propre du présent document avec **pdfLaTeX**, le texte arabe n'a pas été intégré ici. Le contenu correspond à la traduction du résumé français : présentation de l'application PulmoScan AI, de son architecture hors-ligne, des deux modèles d'intelligence artificielle (DenseNet121 et U-Net) et de l'approche de confidentialité des données médicales.

Table des matières

Dédicace	i
Remerciements	ii
Résumé	iii
Abstract	iv
Résumé en arabe	v
Liste des figures	ix
Liste des tableaux	x
Liste des abréviations	xi
Introduction Générale	1
1 Le contexte général du projet	3
Introduction	3
1.1 Contexte et Objectifs	3
1.2 Problématique	4
1.3 Solutions Proposées	4
1.4 Notre Solution Développée	5
1.5 Fonctionnalités de l'Application	5
1.5.1 Écran d'Accueil et Connexion	5
1.5.2 Onboarding	6
1.5.3 Gestion des Patients	6
1.5.4 Création d'Examen	6
1.5.5 Analyse par Intelligence Artificielle (DenseNet121)	6
1.5.6 Segmentation Pulmonaire (U-Net)	6
1.5.7 Visualisation des Résultats	7
1.5.8 Tableau de Bord	7
1.5.9 Exigences Non Fonctionnelles	7
1.6 Étude de l'Existant	7
1.6.1 CheXpert (Stanford University)	7

1.6.2	Infermedica / Ada Health	8
1.6.3	Valeur Ajoutée de PulmoScan AI	8
	Conclusion	8
2	Conception et Modélisation	10
	Introduction	10
2.1	Méthodologies Adoptées	10
2.1.1	Approche Scrum Adaptée	10
2.1.2	Architecture Offline-First	10
2.2	Gestion de Projet	11
2.2.1	Sprints Scrum	11
2.2.2	Diagramme de Gantt	11
2.2.3	Diagramme PERT	12
2.3	Conception	13
2.3.1	Identification des Acteurs	13
2.3.2	Acteurs du Système PulmoScan AI	13
2.3.3	Diagramme de Cas d'Utilisation	13
2.3.4	Diagramme de Classes	14
2.3.5	Diagrammes de Séquence	14
2.3.6	Diagramme d'Activité	17
2.3.7	Diagramme d'État-Transition : Objet Examen	18
2.3.8	Modèle Conceptuel de Données (MCD)	18
2.3.9	Modèle Logique de Données (MLD)	19
	Conclusion	19
3	Implémentation et Déploiement	20
	Introduction	20
3.1	Outils et Technologies Utilisés	20
3.1.1	Frontend Mobile — Flutter & Dart	22
3.1.2	Intelligence Artificielle — TensorFlow Lite	22
3.1.3	Base de Données — SQLite	22
3.1.4	Authentification — SHA-256 & SharedPreferences	23
3.1.5	Backend Optionnel — Node.js / Express / Railway	24
3.1.6	Outils de Développement & DevOps	24
3.2	Structure du Projet Flutter	24
3.3	Pipeline d'Inférence IA Complet	26
3.3.1	Prétraitement des Images	26
3.3.2	Inférence DenseNet121	26
3.3.3	Post-traitement et Sélection du Diagnostic	27
3.3.4	Segmentation U-Net	28

3.4	Sécurité et Confidentialité	28
	Conclusion	28
4	Présentation des Interfaces	30
	Introduction	30
4.1	Écrans d'Onboarding (3 slides)	30
4.2	Landing Page & Connexion	31
4.3	Inscription & Vérification	32
4.4	Dashboard & Statistiques	32
4.5	Liste des Patients	33
4.6	Détails Patient & Historique	34
4.7	Création d'Examen	35
4.8	Résultats de l'Analyse IA	35
4.9	Profil Médecin	36
	Conclusion	37
5	Tests et Assurance Qualité	38
	Introduction	38
5.1	Tests du Modèle IA	38
5.1.1	Validation de la Normalisation (Python)	38
5.1.2	Métriques de Performance par Pathologie	38
5.1.3	Comportement sur Images Synthétiques	39
5.2	Tests Fonctionnels	39
5.3	Tests d'Interface et Ergonomie	40
5.4	Difficultés Rencontrées et Solutions	40
	Conclusion	41
	Conclusion Générale	42
	Webographie / Références	43

Table des figures

2.1	Diagramme de Gantt	12
2.2	Diagramme PERT	12
2.3	Diagramme de cas d'utilisation	13
2.4	Diagramme de classes	14
2.5	Diagramme de séquence : Authentification	15
2.6	Diagramme de séquence : Création Patient	15
2.7	Diagramme de séquence : Acquisition Image	16
2.8	Diagramme de séquence : Analyse IA	17
2.9	Diagramme d'activité	17
2.10	Diagramme d'état-transition : Objet Examen	18
2.11	Modèle Conceptuel de Données (MCD)	18
4.1	Écrans d'onboarding	31
4.2	Landing page et connexion	31
4.3	Inscription et vérification	32
4.4	Dashboard et statistiques	33
4.5	Liste des patients	34
4.6	Détails patient et historique	34
4.7	Création d'examen	35
4.8	Résultats de l'analyse IA	36
4.9	Profil médecin	36

Liste des tableaux

1.1	Exigences non fonctionnelles de PulmoScan AI	7
1.2	Étude comparative de l'existant	8
2.1	Backlog des sprints Scrum	11
3.1	Pile technologique de PulmoScan AI	21
3.2	Inventaire des écrans Flutter	25
3.3	Inventaire des services Flutter	25
3.4	Ordre exact des étiquettes TorchXRayVision (indices 0 à 13)	27
4.1	Patients de démonstration	33
5.1	Validation de la normalisation (tests Python)	38
5.2	AUC par pathologie (* = exclue du diagnostic principal)	39
5.3	Synthèse des tests fonctionnels	40
5.4	Difficultés rencontrées et solutions apportées	41

Liste des abréviations

Abréviation	Signification
AUC	Area Under the Curve (aire sous la courbe ROC)
API	Application Programming Interface
BPCO	Broncho-Pneumopathie Chronique Obstructive
BTS	Brevet de Technicien Supérieur
CIN	Carte d'Identité Nationale
CNN	Convolutional Neural Network (réseau de neurones convolutif)
CRUD	Create, Read, Update, Delete
DAI	Développement des Applications Informatiques
IA	Intelligence Artificielle
JWT	JSON Web Token
MCD	Modèle Conceptuel de Données
MLD	Modèle Logique de Données
NIH	National Institutes of Health
PA	Paquets-Année (tabagisme)
PERT	Program Evaluation and Review Technique
PFE	Projet de Fin d'Études
RGPD	Règlement Général sur la Protection des Données
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
SDK	Software Development Kit
SGBD	Système de Gestion de Base de Données
SHA	Secure Hash Algorithm
SQL	Structured Query Language
TFLite	TensorFlow Lite
TXV	TorchXRayVision
UML	Unified Modeling Language
UI/UX	User Interface / User Experience

Introduction Générale

Les maladies respiratoires figurent parmi les principales causes de morbidité et de mortalité à l'échelle mondiale. Selon l'Organisation mondiale de la santé, des affections telles que la pneumonie, la broncho-pneumopathie chronique obstructive (BPCO), l'emphysème ou les épanchements pleuraux touchent chaque année des centaines de millions de personnes. Au Maroc, les pathologies pulmonaires représentent une charge importante pour le système de santé, particulièrement dans les zones rurales et péri-urbaines où l'accès à un radiologue qualifié demeure limité.

La radiographie thoracique reste l'examen d'imagerie de première intention pour le dépistage de nombreuses pathologies pulmonaires : elle est rapide, peu coûteuse et largement disponible. Néanmoins, son interprétation requiert une expertise spécialisée qui n'est pas toujours présente sur le lieu de soin. Le délai entre l'acquisition de l'image et son interprétation par un spécialiste peut retarder la prise en charge du patient, avec des conséquences cliniques parfois lourdes.

Les progrès récents de l'intelligence artificielle, et plus particulièrement de l'apprentissage profond appliqué à la vision par ordinateur, ont ouvert des perspectives considérables dans l'analyse automatisée des images médicales. Des réseaux de neurones convolutifs, entraînés sur des jeux de données de grande ampleur, atteignent aujourd'hui des performances proches de celles des experts humains pour la détection de certaines pathologies. Toutefois, la majorité des solutions existantes reposent sur une architecture en nuage (*cloud*), ce qui pose des problèmes de confidentialité des données médicales, de dépendance à la connectivité réseau et de coût d'utilisation.

C'est dans ce contexte que s'inscrit notre projet de fin d'études : la conception et le développement de **PulmoScan AI**, une application mobile de pré-diagnostic des pathologies pulmonaires fonctionnant intégralement hors ligne. En embarquant directement sur l'appareil deux modèles d'intelligence artificielle — un réseau DenseNet121 pour la classification multi-pathologie et un réseau U-Net pour la segmentation pulmonaire — l'application offre une assistance au diagnostic immédiate, confidentielle et indépendante de toute infrastructure réseau.

Le présent rapport est organisé en cinq chapitres. Le **premier chapitre** présente le contexte général du projet, sa problématique, les solutions envisagées et l'étude de l'existant. Le **deuxième chapitre** détaille la conception et la modélisation du

système selon une démarche Scrum adaptée, accompagnée des différents diagrammes UML. Le **troisième chapitre** décrit l'implémentation technique, les outils utilisés et le pipeline d'inférence de l'intelligence artificielle. Le **quatrième chapitre** présente les interfaces utilisateur réalisées. Enfin, le **cinquième chapitre** expose les tests menés et les démarches d'assurance qualité. Une conclusion générale et des perspectives clôturent ce travail.

Chapitre 1

Le contexte général du projet

Introduction

Ce premier chapitre a pour objectif de poser les fondations du projet PulmoScan AI. Nous y présentons d’abord le contexte général et les objectifs poursuivis, puis nous formulons la problématique à laquelle l’application entend répondre. Nous examinons ensuite les différentes solutions envisageables, avant de décrire en détail la solution que nous avons développée ainsi que l’ensemble de ses fonctionnalités. Le chapitre se termine par une étude de l’existant, qui situe notre application par rapport aux solutions comparables du marché et met en lumière sa valeur ajoutée.

1.1 Contexte et Objectifs

Le projet PulmoScan AI s’inscrit dans le cadre du projet de fin d’études du cycle BTS en Développement des Applications Informatiques, au sein du Centre de Formation Al Kendi à Casablanca. Il vise à concevoir une application mobile capable d’assister le personnel médical — médecins généralistes, internes, infirmiers spécialisés — dans l’interprétation préliminaire des radiographies thoraciques.

L’idée centrale du projet est de mettre la puissance de l’intelligence artificielle au service de la première ligne de soins, là où l’expertise radiologique fait souvent défaut. En automatisant la détection de signes pathologiques sur les radiographies, l’application permet de hiérarchiser les cas, d’orienter rapidement les patients présentant des anomalies préoccupantes et de documenter chaque examen de manière structurée.

Les objectifs principaux assignés à PulmoScan AI sont les suivants :

- fournir un outil de **pré-diagnostic** rapide des pathologies pulmonaires à partir d’une radiographie thoracique ;
- garantir un fonctionnement **100% hors ligne**, sans dépendance à un serveur distant ni à une connexion internet ;
- assurer la **confidentialité** des données médicales en les conservant exclusivement sur l’appareil ;
- offrir une **interface en français**, claire et ergonomique, adaptée au contexte marocain ;

- permettre la **gestion complète des patients et des examens** (dossier médical, historique, antécédents) ;
- générer des **rapports PDF** exploitables et partageables.

Il convient de souligner d'emblée que PulmoScan AI est conçu comme un *outil d'aide à la décision* et non comme un dispositif de diagnostic autonome. Le jugement final demeure toujours celui du praticien.

1.2 Problématique

La problématique à laquelle répond PulmoScan AI peut être formulée ainsi : *comment rendre l'analyse assistée par intelligence artificielle des radiographies thoraciques accessible, rapide et confidentielle, y compris dans des contextes dépourvus de connectivité fiable et d'expertise radiologique ?*

Plusieurs contraintes structurent cette problématique. D'abord, la **disponibilité** : dans de nombreuses structures de soins, l'interprétation d'une radiographie par un radiologue peut prendre des heures, voire des jours. Ensuite, la **connectivité** : les solutions reposant sur le cloud sont inutilisables dans les zones où la couverture réseau est insuffisante ou intermittente. Par ailleurs, la **confidentialité** : le transfert de données médicales vers des serveurs distants, souvent situés à l'étranger, soulève des questions juridiques et éthiques majeures. Enfin, le **coût** : les API d'analyse d'images médicales en nuage sont fréquemment payantes et facturées à l'usage, ce qui les rend inadaptées à un déploiement à grande échelle dans des contextes à ressources limitées.

PulmoScan AI cherche à lever simultanément ces quatre contraintes en embarquant l'intégralité de la chaîne de traitement directement sur l'appareil mobile.

1.3 Solutions Proposées

Face à cette problématique, trois grandes familles de solutions étaient envisageables.

La **première approche** consiste à recourir à une solution en nuage, où l'image est transmise à un serveur distant qui exécute l'inférence et renvoie le résultat. Cette approche bénéficie d'une puissance de calcul élevée et de modèles très volumineux, mais elle se heurte aux problèmes de connectivité, de confidentialité et de coût évoqués précédemment.

La **deuxième approche** repose sur une architecture hybride, où une partie du traitement est effectuée localement et une autre déléguée au serveur. Si elle offre un compromis intéressant, elle conserve une dépendance partielle au réseau et complexifie considérablement l'architecture logicielle.

La **troisième approche**, que nous avons retenue, est une architecture entièrement embarquée (*on-device*). Les modèles d’intelligence artificielle sont convertis au format TensorFlow Lite, optimisés pour les contraintes des appareils mobiles, puis exécutés localement. Cette approche maximise la confidentialité et la disponibilité, au prix d’un travail d’optimisation des modèles pour les faire tenir dans une empreinte mémoire raisonnable.

1.4 Notre Solution Développée

PulmoScan AI est une application mobile développée avec le framework **Flutter** (langage Dart), permettant un déploiement multiplateforme à partir d’une base de code unique. L’application s’articule autour de deux modèles d’intelligence artificielle embarqués via **TensorFlow Lite** :

- un modèle de **classification DenseNet121** (13,4 Mo) issu de la bibliothèque TorchXRayVision, qui produit des scores pour 14 pathologies pulmonaires ;
- un modèle de **segmentation U-Net** (29,6 Mo) qui isole les champs pulmonaires et génère un masque binaire superposable à l’image.

Les données — utilisateurs, patients, examens et résultats — sont persistées dans une base **SQLite** locale (version 4 du schéma), tandis que l’authentification est assurée par un hachage **SHA-256** des mots de passe et une session stockée via **SharedPreferences**. L’ensemble fonctionne sans connexion. Un backend optionnel (Node.js / Express) peut être activé pour la synchronisation, mais il n’est jamais requis pour le fonctionnement nominal.

1.5 Fonctionnalités de l’Application

Cette section décrit les principales fonctionnalités offertes par PulmoScan AI, organisées selon le parcours type d’un utilisateur.

1.5.1 Écran d’Accueil et Connexion

À son lancement, l’application présente un écran d’accueil (*landing*) mettant en valeur l’identité visuelle de PulmoScan AI et ses fonctionnalités clés. Cet écran propose un défilement animé conduisant le médecin vers le formulaire de connexion. L’authentification s’effectue par couple e-mail / mot de passe, le mot de passe étant vérifié contre son empreinte SHA-256 stockée localement. Un compte de démonstration (`demo@pulmoscan.ma` / `Demo1234`) permet d’explorer l’application sans inscription préalable.

1.5.2 Onboarding

Lors de la première utilisation, un parcours d'introduction (*onboarding*) composé de trois écrans présente successivement les trois piliers de l'application : la **Capture** de la radiographie, l'**Analyse par IA** et la génération du **Rapport**. Ce parcours, animé par le widget personnalisé **ApertureMark**, familiarise rapidement l'utilisateur avec la philosophie de l'outil.

1.5.3 Gestion des Patients

L'application permet la création, la consultation, la modification et la suppression des dossiers patients. Chaque patient est caractérisé par son nom, son âge, son genre, son adresse e-mail, son numéro de téléphone, son numéro de carte d'identité nationale (CIN), ses antécédents médicaux et sa date de naissance. Une liste consultable et filtrable affiche l'ensemble des patients, avec des badges de statut, et un écran de détail présente le dossier complet ainsi que l'historique des examens.

1.5.4 Création d'Examen

La création d'un examen suit un assistant en plusieurs étapes : sélection du patient concerné, acquisition de l'image radiographique (depuis la galerie ou l'appareil photo via le package `image_picker`), puis lancement de l'analyse. Chaque examen est rattaché à un patient et porte un statut évolutif (`en_attente`, `en_cours`, `termine`).

1.5.5 Analyse par Intelligence Artificielle (DenseNet121)

Le cœur de l'application réside dans l'analyse par le modèle DenseNet121. L'image acquise est prétraitée (recadrage carré centré, redimensionnement à 224×224 , conversion en niveaux de gris et normalisation), puis soumise au modèle qui produit un vecteur de 18 sorties sigmoïdes. Les 14 premières correspondent aux pathologies NIH. Un post-traitement sélectionne le diagnostic dominant en excluant quatre classes ambiguës à faible AUC, calcule un indice de confiance et détermine un niveau de sévérité.

1.5.6 Segmentation Pulmonaire (U-Net)

En parallèle de la classification, le modèle U-Net segmente les champs pulmonaires. L'image est redimensionnée à 256×256 , normalisée puis traitée par le réseau qui produit une carte de probabilités, binarisée à un seuil de 0,5. Le masque résultant est superposé à l'image originale, permettant de visualiser la région anatomique analysée.

1.5.7 Visualisation des Résultats

L'écran de résultats présente le diagnostic principal accompagné de son indice de confiance et de son niveau de sévérité, un histogramme des scores des 14 pathologies, ainsi que la superposition du masque de segmentation. Le médecin peut consulter ces éléments, y ajouter des notes cliniques et générer un rapport PDF.

1.5.8 Tableau de Bord

Le tableau de bord (*dashboard*) offre une vue synthétique de l'activité : cartes statistiques (nombre de patients, d'examens, répartition par sévérité), liste des examens récents et fiches d'information sur les modèles d'IA, accessibles d'une simple tape.

1.5.9 Exigences Non Fonctionnelles

Au-delà des fonctionnalités, PulmoScan AI répond à un ensemble d'exigences non fonctionnelles essentielles, résumées dans le tableau 1.1.

TABLE 1.1 – Exigences non fonctionnelles de PulmoScan AI

Exigence	Description
Performance	Inférence DenseNet121 et U-Net en quelques secondes sur appareil mobile standard.
Disponibilité	Fonctionnement intégral hors ligne, sans dépendance réseau.
Confidentialité	Données médicales conservées exclusivement sur l'appareil.
Sécurité	Mots de passe hachés en SHA-256, session protégée.
Ergonomie	Interface en français, design system cohérent, navigation intuitive.
Portabilité	Code Flutter unique déployable sur Android et iOS.
Maintenabilité	Architecture en services (singletons) et séparation des responsabilités.

1.6 Étude de l'Existant

Pour situer PulmoScan AI, nous avons étudié deux solutions de référence dans le domaine de l'analyse médicale assistée par IA.

1.6.1 CheXpert (Stanford University)

CheXpert est un système développé par l'université de Stanford, reposant sur un vaste jeu de données de radiographies thoraciques. Ses modèles atteignent des performances remarquables (AUC supérieures à 0,90 pour plusieurs pathologies). Cependant,

CheXpert est principalement une solution en nuage : l'analyse nécessite l'envoi des images vers des serveurs distants, généralement situés aux États-Unis. Cette architecture pose un problème de conformité au regard de la protection des données (RGPD), l'API associée est payante, l'interface n'est pas francophone et il n'existe pas d'application mobile native grand public.

1.6.2 Infermedica / Ada Health

Infermedica et Ada Health sont des solutions de pré-diagnostic basées sur l'analyse des symptômes déclarés par l'utilisateur, via un agent conversationnel (chatbot). Elles ne traitent pas d'images radiographiques et ne fournissent donc pas de diagnostic d'imagerie. Leur portée est différente de celle de PulmoScan AI : elles s'adressent au triage symptomatique, ne fonctionnent pas hors ligne et ne produisent pas de détection pathologique précise sur image.

1.6.3 Valeur Ajoutée de PulmoScan AI

Le tableau 1.2 synthétise la comparaison. PulmoScan AI se distingue par son fonctionnement intégralement hors ligne, la conservation des données sur l'appareil (gage de confidentialité et de meilleure conformité), son interface française, sa double intelligence artificielle (classification *et* segmentation) et sa couverture de 14 pathologies issues du jeu NIH ChestX-ray14.

TABLE 1.2 – Étude comparative de l'existant

Critère	CheXpert	Ada Health	PulmoScan AI
Analyse de radiographie	Oui	Non	Oui
Fonctionnement hors ligne	Non	Non	Oui
Données locales (confidentialité)	Non	Non	Oui
Interface française	Non	Partielle	Oui
Segmentation pulmonaire	Non	Non	Oui
Mobile natif	Non	Oui	Oui
Coût d'utilisation	Payant	Freemium	Gratuit

Conclusion

Ce premier chapitre a permis de cerner le contexte, la problématique et les objectifs du projet PulmoScan AI, ainsi que de positionner notre solution par rapport aux alternatives existantes. Il ressort que le caractère hors ligne et la confidentialité des données

constituent les principaux atouts différenciants de notre application. Le chapitre suivant détaille la démarche de conception et de modélisation adoptée pour concrétiser ces objectifs.

Chapitre 2

Conception et Modélisation

Introduction

Après avoir défini le contexte et les objectifs du projet, ce deuxième chapitre est consacré à la conception et à la modélisation de PulmoScan AI. Nous y présentons d’abord les méthodologies adoptées, puis la gestion du projet à travers les sprints, le diagramme de Gantt et le diagramme PERT. Nous abordons ensuite la conception détaillée du système au moyen de diagrammes UML — cas d’utilisation, classes, séquence, activité, état-transition — avant de présenter les modèles de données conceptuel et logique.

2.1 Méthodologies Adoptées

2.1.1 Approche Scrum Adaptée

Pour piloter le développement de PulmoScan AI, nous avons adopté une démarche agile inspirée de Scrum, adaptée à la taille de notre binôme et à la durée du projet. Le travail a été découpé en **sprints** d’une à deux semaines, chacun aboutissant à un incrément fonctionnel testable. À l’issue de chaque sprint, une revue informelle permettait de valider les fonctionnalités livrées et d’ajuster le *backlog* restant. Cette approche itérative s’est révélée particulièrement adaptée à un projet comportant une forte composante d’expérimentation, notamment pour la mise au point du pipeline d’inférence de l’IA.

2.1.2 Architecture Offline-First

Le choix structurant de l’architecture est le paradigme *offline-first*. Contrairement à une architecture centrée sur le réseau, où les données et les traitements résident sur un serveur, l’architecture offline-first place l’appareil mobile au centre du dispositif : les modèles, la base de données et toute la logique applicative y sont embarqués. Le réseau n’intervient, le cas échéant, que pour une synchronisation optionnelle. Cette architecture garantit la disponibilité permanente de l’application, la confidentialité des données et l’absence de latence réseau lors de l’inférence.

2.2 Gestion de Projet

2.2.1 Sprints Scrum

Le développement s'est échelonné sur cinq sprints, dont le contenu est détaillé dans le tableau 2.1.

TABLE 2.1 – Backlog des sprints Scrum

Sprint	Durée	Contenu
Sprint 1	2 semaines	Configuration du projet, authentification, CRUD patients, base SQLite v1.
Sprint 2	2 semaines	Import d'image, pipeline TensorFlow Lite, intégration du modèle DenseNet121.
Sprint 3	2 semaines	Écran de résultats, segmentation U-Net, histogramme des 14 pathologies.
Sprint 4	2 semaines	Tableau de bord, onboarding, design system V4, données de démonstration.
Sprint 5	1 semaine	Corrections de bugs, fix de l'ordre des labels, fix de l'overflow Android, tests finaux.

2.2.2 Diagramme de Gantt

Le diagramme de Gantt (figure 2.1) présente l'ordonnancement temporel des sprints et leur répartition sur la durée du projet.



FIGURE 2.1 – Diagramme de Gantt

2.2.3 Diagramme PERT

Le diagramme PERT (figure 2.2) modélise les dépendances entre les tâches du projet et met en évidence le chemin critique.



FIGURE 2.2 – Diagramme PERT

2.3 Conception

2.3.1 Identification des Acteurs

Un acteur, au sens de la modélisation UML, désigne une entité externe (humaine ou logicielle) qui interagit avec le système. L'identification des acteurs constitue la première étape de l'analyse fonctionnelle, car elle délimite le périmètre du système et oriente la définition des cas d'utilisation.

2.3.2 Acteurs du Système PulmoScan AI

Le système PulmoScan AI met en jeu les acteurs suivants :

- le **Médecin** (acteur principal) : il s'authentifie, gère les patients, crée des examens, lance les analyses, consulte les résultats et génère les rapports ;
- le **Moteur d'IA** (acteur secondaire système) : il exécute l'inférence des modèles DenseNet121 et U-Net ;
- le **Backend optionnel** (acteur secondaire système) : il assure, lorsqu'il est activé, la synchronisation des données via une API REST.

2.3.3 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation (figure 2.3) recense les principales fonctionnalités offertes au médecin et leurs relations.



FIGURE 2.3 – Diagramme de cas d'utilisation

2.3.4 Diagramme de Classes

Le diagramme de classes (figure 2.4) décrit la structure statique du système : les classes métier (Utilisateur, Patient, Examen, Résultat), leurs attributs, leurs méthodes et leurs associations.



FIGURE 2.4 – Diagramme de classes

2.3.5 Diagrammes de Séquence

Les diagrammes de séquence illustrent les interactions dynamiques entre les acteurs et les composants du système au fil du temps, pour les scénarios les plus représentatifs.

Séquence Authentification

La figure 2.5 décrit le scénario d'authentification : saisie des identifiants, hachage SHA-256, vérification en base SQLite et création de la session.



FIGURE 2.5 – Diagramme de séquence : Authentification

Séquence Création Patient

La figure 2.6 décrit la création d'un patient : saisie du formulaire, validation des champs et insertion en base.

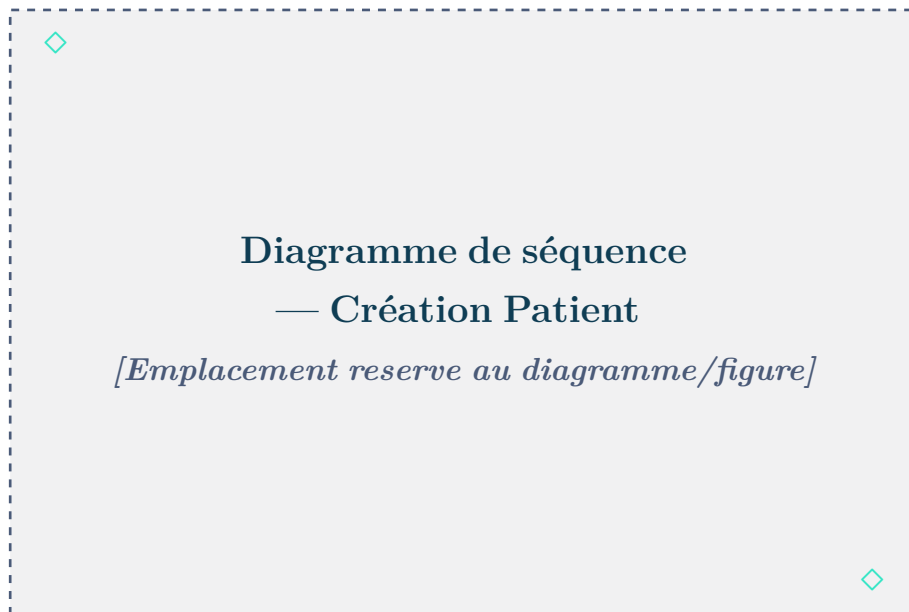


FIGURE 2.6 – Diagramme de séquence : Création Patient

Séquence Acquisition Image

La figure 2.7 décrit l'acquisition de l'image radiographique via `image_picker` et son enregistrement.



FIGURE 2.7 – Diagramme de séquence : Acquisition Image

Séquence Analyse IA

La figure 2.8 décrit le scénario d'analyse : prétraitement de l'image, inférence DenseNet121, post-traitement, segmentation U-Net et enregistrement du résultat.

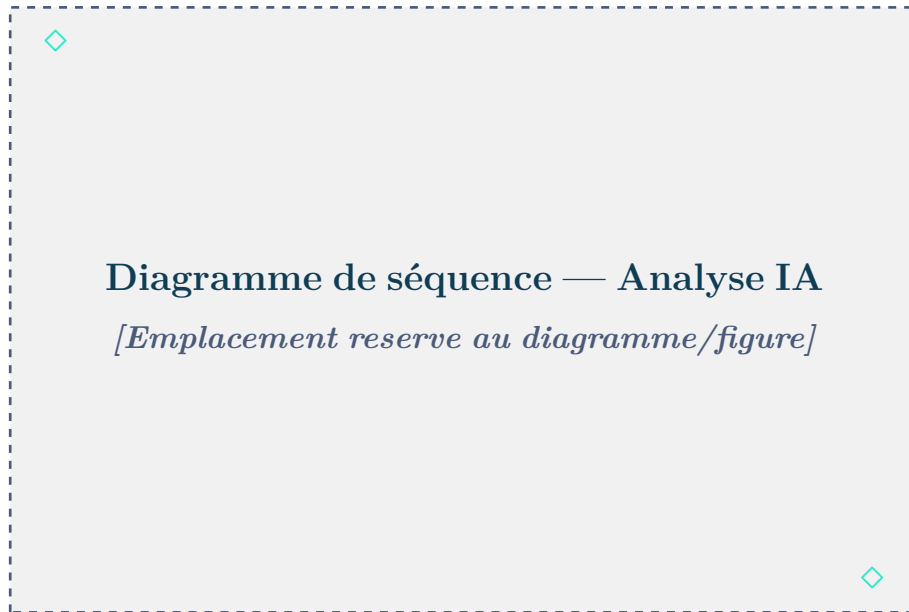


FIGURE 2.8 – Diagramme de séquence : Analyse IA

2.3.6 Diagramme d'Activité

Le diagramme d'activité (figure 2.9) modélise le flux complet d'un examen, depuis la sélection du patient jusqu'à la génération du rapport.



FIGURE 2.9 – Diagramme d'activité

2.3.7 Diagramme d’État-Transition : Objet Examen

La figure 2.10 présente le cycle de vie d’un examen à travers ses états successifs : `en_attente`, `en_cours` et `termine`.



FIGURE 2.10 – Diagramme d’état-transition : Objet Examen

2.3.8 Modèle Conceptuel de Données (MCD)

Le modèle conceptuel de données (figure 2.11) décrit les entités du domaine, leurs propriétés et les associations qui les relient, indépendamment de toute implémentation technique.

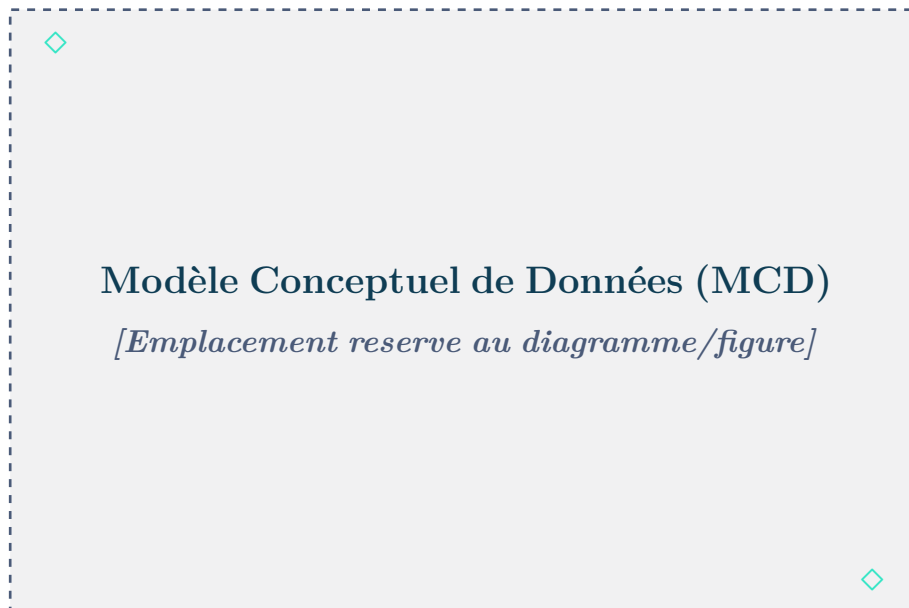


FIGURE 2.11 – Modèle Conceptuel de Données (MCD)

2.3.9 Modèle Logique de Données (MLD)

Le passage du MCD au modèle logique de données traduit les entités en relations et les associations en clés étrangères. Le MLD de PulmoScan AI comporte quatre relations :

```
users(id, name, email, password, role, created_at)
patients(id, nom, age, genre, email, telephone, cin, antecedents, date_naissance,
date_creation)
examens(id, #patient_id, image_path, notes, date_examen, statut)
resultats(id, #exam_id, diagnostic, confidence, severite, details, date_analyse)
```

où la clé primaire est soulignée et la clé étrangère préfixée d'un dièse. La relation **examens** référence **patients** par **patient_id**, et la relation **resultats** référence **examens** par **exam_id**. On obtient ainsi une chaîne relationnelle 1 : N entre un patient, ses examens et les résultats associés.

Conclusion

Ce chapitre a formalisé la conception de PulmoScan AI : méthodologie agile, architecture offline-first, modélisation UML complète et structuration des données. Ces fondations conceptuelles guident désormais la phase d'implémentation, détaillée dans le chapitre suivant.

Chapitre 3

Implémentation et Déploiement

Introduction

Ce troisième chapitre décrit la réalisation technique de PulmoScan AI. Nous présentons d’abord l’ensemble des outils et technologies mobilisés, puis la structure du projet Flutter. Nous détaillons ensuite le pipeline complet d’inférence de l’intelligence artificielle — du prétraitement de l’image à la segmentation U-Net — en nous appuyant sur des extraits de code commentés. Le chapitre se conclut par les mesures de sécurité et de confidentialité mises en œuvre.

3.1 Outils et Technologies Utilisés

Le tableau 3.1 synthétise la pile technologique du projet.

TABLE 3.1 – Pile technologique de PulmoScan AI

Couche	Technologie	Rôle
Frontend mobile	Flutter / Dart	Interface et logique applicative multiplateforme.
Intelligence artificielle	TensorFlow Lite (<code>tfmlite_flutter</code> 0.12.0)	Inférence embarquée des modèles.
Traitement d'image	<code>image</code> 4.1.7	Recadrage, redimensionnement, conversion.
Base de données	SQLite (<code>sqflite</code> 2.3.0)	Persistance locale des données.
Authentification	<code>crypto</code> 3.0.3 (SHA-256)	Hachage des mots de passe.
Session	<code>shared_preferences</code> 2.3.0	Stockage de la session utilisateur.
Acquisition image	<code>image_picker</code> 1.0.7	Galerie / appareil photo.
Rapports	<code>pdf</code> 3.11.0 + <code>printing</code> 5.13.0	Génération et impression de PDF.
Typographie	<code>google_fonts</code> 6.2.1	Polices de l'interface.
Internationalisation	<code>intl</code> 0.20.0	Formatage des dates et nombres.
Réseau (optionnel)	<code>http</code> 1.2.0	Client REST vers le backend.
Backend optionnel	Node.js / Express / Railway	API REST et JWT.

3.1.1 Frontend Mobile — Flutter & Dart

Flutter est un framework open-source développé par Google permettant de créer des applications natives compilées pour mobile, web et bureau à partir d'une base de code unique écrite en Dart. Son moteur de rendu propre garantit une interface fluide et cohérente sur toutes les plateformes. Nous avons retenu Flutter pour sa productivité, son écosystème de packages riche et la qualité de son intégration avec TensorFlow Lite. Le projet exploite par ailleurs `flutter_localizations` pour la prise en charge de la langue française et `cupertino_icons` pour les icônes.

3.1.2 Intelligence Artificielle — TensorFlow Lite

TensorFlow Lite est la déclinaison légère de TensorFlow, conçue pour l'inférence sur appareils mobiles et embarqués. Les modèles entraînés sont convertis au format `.tflite`, optimisé pour une empreinte mémoire et une latence réduites. Le package `tflite_flutter` expose l'interpréteur natif à Dart, permettant de charger les modèles et d'exécuter l'inférence directement sur l'appareil.

3.1.3 Base de Données — SQLite

SQLite est un moteur de base de données relationnelle léger, sans serveur, intégré à l'application. Via le package `sqflite` (et `path` pour la localisation du fichier), PulmoScan AI gère une base locale (version 4 du schéma) comportant quatre tables. Le schéma complet est présenté dans le listing 3.1.

```
1 CREATE TABLE users (  
2   id INTEGER PRIMARY KEY AUTOINCREMENT,  
3   name TEXT NOT NULL,  
4   email TEXT NOT NULL UNIQUE,  
5   password TEXT NOT NULL,          -- empreinte SHA-256  
6   role TEXT DEFAULT 'medecin',  
7   created_at TEXT NOT NULL  
8 );  
9  
10 CREATE TABLE patients (  
11   id INTEGER PRIMARY KEY AUTOINCREMENT,  
12   nom TEXT NOT NULL,  
13   age INTEGER NOT NULL,  
14   genre TEXT NOT NULL,  
15   email TEXT NOT NULL UNIQUE,  
16   telephone TEXT DEFAULT '',  
17   cin TEXT NOT NULL UNIQUE,  
18   antecedents TEXT DEFAULT '',  
19   date_naissance TEXT,
```

```

20   date_creation TEXT NOT NULL
21 );
22
23 CREATE TABLE examens (
24   id INTEGER PRIMARY KEY AUTOINCREMENT,
25   patient_id INTEGER NOT NULL,
26   image_path TEXT NOT NULL,
27   notes TEXT DEFAULT '',
28   date_examen TEXT NOT NULL,
29   statut TEXT DEFAULT 'en_attente', -- en_attente / en_cours /
    termine
30   FOREIGN KEY (patient_id) REFERENCES patients (id)
31 );
32
33 CREATE TABLE resultats (
34   id INTEGER PRIMARY KEY AUTOINCREMENT,
35   exam_id INTEGER NOT NULL,
36   diagnostic TEXT NOT NULL,
37   confidence REAL NOT NULL,          -- 0.0 a 1.0, affiche x100
38   severite TEXT NOT NULL,           -- urgent/moyen/normal/modere/
    severe/faible
39   details TEXT DEFAULT '',
40   date_analyse TEXT NOT NULL,
41   FOREIGN KEY (exam_id) REFERENCES examens (id)
42 );

```

Listing 3.1 – Schéma SQL de la base SQLite (version 4)

3.1.4 Authentification — SHA-256 & SharedPreferences

L'authentification de PulmoScan AI est intégralement locale. Lors de l'inscription, le mot de passe de l'utilisateur est haché avec l'algorithme SHA-256 fourni par le package `crypto`, puis seule l'empreinte est stockée en base. Lors de la connexion, le mot de passe saisi est haché de la même manière et comparé à l'empreinte enregistrée. La session est conservée dans `SharedPreferences` (clés `user_name`, `user_email`, `user_role`, `is_logged_in`). Le listing 3.2 illustre le hachage.

```

1  import 'package:crypto/crypto.dart';
2  import 'dart:convert';
3
4  String hashPassword(String password) {
5    return sha256.convert(utf8.encode(password)).toString();
6  }

```

Listing 3.2 – Hachage SHA-256 du mot de passe

3.1.5 Backend Optionnel — Node.js / Express / Railway

Un backend optionnel, développé avec Node.js et Express et déployable sur la plateforme Railway, peut être activé pour la synchronisation des données. Le client `ApiService` s’y connecte par défaut sur `localhost:3000/api` via la redirection ADB (`adb reverse tcp:3000 tcp:3000`), et l’authentification y repose sur un jeton JWT stocké dans `SharedPreferences`. Les points d’accès couvrent l’authentification (`POST /auth/login`, `POST /auth/register`), la gestion des patients (`GET/POST/PUT/DELETE /patients`), les examens (`GET/POST /examens`) et l’analyse (`POST /examens/:id/analyse`). Ce backend n’est jamais requis pour le fonctionnement nominal : toutes les fonctionnalités essentielles opèrent hors ligne.

3.1.6 Outils de Développement & DevOps

Le développement a mobilisé l’éditeur Visual Studio Code avec les extensions Flutter et Dart, le système de gestion de versions Git, les notebooks Google Colab pour l’entraînement et la conversion des modèles, ainsi que l’outil `adb` pour le déploiement et le débogage sur appareil physique. Les modèles ont été convertis en `.tflite` depuis PyTorch, puis intégrés aux ressources de l’application.

3.2 Structure du Projet Flutter

Le projet Flutter est organisé selon une séparation claire des responsabilités : les écrans (*screens*), les services métier (*services*), le thème et les widgets réutilisables. On dénombre 13 écrans et 7 services principaux. Le tableau 3.2 et le tableau 3.3 en présentent l’inventaire.

TABLE 3.2 – Inventaire des écrans Flutter

Fichier	Lignes	Rôle
landing_screen.dart	827	Hero animé et défilement vers le formulaire de connexion.
onboarding.dart	304	Trois slides (Capture / IA / Rapport), widget ApertureMark.
login.dart	687	Connexion e-mail/mot de passe, SHA-256, session.
register.dart	506	Formulaire d'inscription.
verify_screen.dart	290	Vérification d'un code à 6 chiffres.
dashboard.dart	707	Cartes statistiques, examens récents, fiches modèles IA.
patients_list_screen.dart	367	Liste filtrable, badges de statut.
add_patient_screen.dart	423	Formulaire de création de patient avec validation.
patient_detail_screen.dart	506	Dossier patient et historique des examens.
exam_screen.dart	568	Assistant : patient → image → IA.
results_screen.dart	1226	Diagnostic principal, histogramme 14 barres, masque U-Net.
history_screen.dart	266	Historique global des examens.
profile_screen.dart	647	Profil médecin, réglages, changement de mot de passe.

TABLE 3.3 – Inventaire des services Flutter

Service	Lignes	Rôle
ai_service.dart	218	Singleton d'inférence TFLite (DenseNet121 + U-Net).
database_service.dart	508	Singleton CRUD, SQLite v4.
auth_service.dart	116	Authentification SHA-256 locale et session.
api_service.dart	174	Client REST optionnel (JWT, localhost :3000/api).
pdf_service.dart	682	Génération des rapports PDF.
segmentation_service.dart	89	Génération de la superposition du masque U-Net.
sync_service.dart	11	Stub de synchronisation cloud future.

3.3 Pipeline d’Inférence IA Complet

Le pipeline d’inférence constitue le cœur technique de PulmoScan AI. Il se décompose en quatre phases : le prétraitement de l’image, l’inférence DenseNet121, le post-traitement et la segmentation U-Net.

3.3.1 Prétraitement des Images

Le prétraitement vise à transformer l’image radiographique fournie par l’utilisateur en un tenseur conforme aux attentes du modèle DenseNet121, à savoir un tableau de dimensions $[1, 224, 224, 1]$ en virgule flottante, en niveaux de gris et normalisé dans l’intervalle $[0, 1]$. L’image subit d’abord un recadrage carré centré, puis un redimensionnement à 224×224 , et enfin une conversion en luminance selon la recommandation Rec. 601, suivie d’une division par 255. Le listing 3.3 présente le code Dart correspondant.

```

1 // Recadrage carre centre
2 final side = w < h ? w : h;
3 final cropped = img.copyCrop(decoded,
4   x: (w - side) ~/ 2, y: (h - side) ~/ 2, width: side, height: side
5   );
6 // Redimensionnement
7 final resized = img.copyResize(cropped, width: 224, height: 224);
8
9 // Luminance Rec. 601 -> [0,1]
10 final lum = 0.299 * pixel.r + 0.587 * pixel.g + 0.114 * pixel.b;
11 inputData[idx++] = lum / 255.0;

```

Listing 3.3 – Prétraitement de l’image (Dart)

Point critique : normalisation

Le modèle DenseNet121 issu de TorchXRayVision intègre déjà (*baked in*) sa propre normalisation interne. Lui fournir des valeurs dans l’intervalle TorchXRayVision $[-1024, 1024]$ provoque la **saturation de toutes les sorties**. Seule une normalisation par division par 255 (intervalle $[0, 1]$) produit des sorties exploitables. Ce point, confirmé par les tests Python, est développé au chapitre 5.

3.3.2 Inférence DenseNet121

Une fois le tenseur d’entrée préparé, il est soumis à l’interpréteur TensorFlow Lite chargé avec le fichier `pulmoscan_model.tflite` (13,4 Mo). Le modèle produit un vecteur de sortie $[1, 18]$ de valeurs sigmoïdes. Les 14 premiers indices correspondent aux

pathologies NIH ; les indices 14 à 17 restent invariablement aux alentours de 0,5 car ils ne sont pas entraînés. Un point d’attention majeur concerne l’ordre des étiquettes : l’ordre TorchXRayVision n’est **pas alphabétique**. Le tableau 3.4 présente l’ordre exact, indispensable à une interprétation correcte.

TABLE 3.4 – Ordre exact des étiquettes TorchXRayVision (indices 0 à 13)

Idx	Pathologie	Idx	Pathologie
0	Atelectasis	7	Effusion
1	Consolidation	8	Pneumonia
2	Infiltration	9	Pleural_Thickening
3	Pneumothorax	10	Cardiomegaly
4	Edema	11	Nodule
5	Emphysema	12	Mass
6	Fibrosis	13	Hernia

3.3.3 Post-traitement et Sélection du Diagnostic

Le post-traitement sélectionne la pathologie dominante. Quatre classes jugées ambiguës, en raison d’une AUC inférieure à 0,80 (Infiltration, Pneumonia, Nodule, Consolidation), sont exclues du diagnostic principal — tout en restant affichées dans l’histogramme. Le score brut le plus élevé parmi les classes restantes détermine le diagnostic. En l’absence de score suffisant ($< 0,04$), un repli sélectionne le score le plus élevé toutes classes confondues. L’indice de confiance est ensuite calculé par $(\text{bestRaw} \times 2,5)$ borné à l’intervalle $[0,55; 0,97]$, et la sévérité est fixée à **urgent** si le score brut atteint 0,20, à **moyen** sinon. Le listing 3.4 détaille cette logique. Des seuils par classe sont également définis (Atelectasis 0,07 ; Cardiomegaly 0,06 ; Effusion 0,08 ; Emphysema 0,05 ; Fibrosis 0,04 ; etc.).

```

1  const ambiguous = ['Infiltration', 'Pneumonia', 'Nodule', '
    Consolidation'];
2  int bestIdx = -1;
3  double bestRaw = 0;
4
5  for (int i = 0; i < labels.length; i++) {
6      if (!ambiguous.contains(labels[i]) && raw[i] > bestRaw) {
7          bestRaw = raw[i];
8          bestIdx = i;
9      }
10 }
11
12 if (bestIdx < 0 || bestRaw < 0.04) { // repli : meilleur score
    global

```

```
13   for (int i = 0; i < labels.length; i++) {  
14       if (raw[i] > bestRaw) { bestRaw = raw[i]; bestIdx = i; }  
15   }  
16 }  
17  
18 final confidence = (bestRaw * 2.5).clamp(0.55, 0.97);  
19 final severite = bestRaw >= 0.20 ? 'urgent' : 'moyen';
```

Listing 3.4 – Post-traitement et sélection du diagnostic (Dart)

3.3.4 Segmentation U-Net

En parallèle de la classification, le modèle U-Net (`unet_lung.tflite`, 29,6 Mo, non quantifié) segmente les champs pulmonaires. Son entrée est un tenseur $[1, 256, 256, 1]$ en niveaux de gris normalisé par division par 255. Sa sortie est une carte de probabilités $[1, 256, 256, 1]$, binarisée à un seuil de 0,5 pour produire un masque. Ce masque, généré par le `segmentation_service`, est ensuite superposé à l'image originale dans l'écran de résultats, offrant au médecin une visualisation claire de la région anatomique prise en compte par l'analyse.

3.4 Sécurité et Confidentialité

La sécurité et la confidentialité constituent des préoccupations centrales d'une application traitant des données médicales. PulmoScan AI met en œuvre plusieurs mesures. D'abord, les mots de passe ne sont jamais stockés en clair : seules leurs empreintes SHA-256 sont conservées. Ensuite, l'intégralité des données — patients, examens, résultats — reste sur l'appareil, dans la base SQLite locale, sans transfert vers un serveur distant lors du fonctionnement nominal. Cette localisation des données réduit considérablement la surface d'exposition et améliore la conformité aux exigences de protection des données personnelles. Enfin, lorsque le backend optionnel est activé, l'authentification s'appuie sur des jetons JWT, et les communications transitent par un canal contrôlé. La combinaison de ces mesures fait de la confidentialité non pas une option, mais une propriété structurelle de l'architecture.

Conclusion

Ce chapitre a détaillé la réalisation technique de PulmoScan AI : la pile technologique, la structure du projet, le pipeline d'inférence des deux modèles d'IA et les mesures de sécurité. La maîtrise fine de la normalisation et de l'ordre des étiquettes

s'est révélée déterminante pour la justesse des prédictions. Le chapitre suivant présente les interfaces utilisateur qui exposent ces traitements.

Chapitre 4

Présentation des Interfaces

Introduction

Ce chapitre présente les interfaces utilisateur de PulmoScan AI, conçues selon le design system V4. Ce système repose sur un fond sombre (#0A0F1A), des surfaces de cartes (#131B2E), un accent principal turquoise (#34E5C5) et une hiérarchie de teintes de texte. Trois couleurs sémantiques signalent la sévérité : rouge pour l'urgence, ambre pour les cas modérés et turquoise pour les cas normaux. Le widget animé **ApertureMark** incarne l'identité visuelle de l'application. Nous parcourons ci-après les principaux écrans.

4.1 Écrans d'Onboarding (3 slides)

Le parcours d'introduction se compose de trois écrans successifs présentant les trois piliers de PulmoScan AI : la **Capture** de la radiographie, l'**Analyse par IA** et la production du **Rapport**. Chaque slide associe une illustration animée par le widget **ApertureMark** à un texte concis. Un indicateur de progression et un bouton de navigation permettent à l'utilisateur de parcourir ou de passer cette introduction. Une attention particulière a été portée à l'adaptation à la barre de navigation gestuelle d'Android.

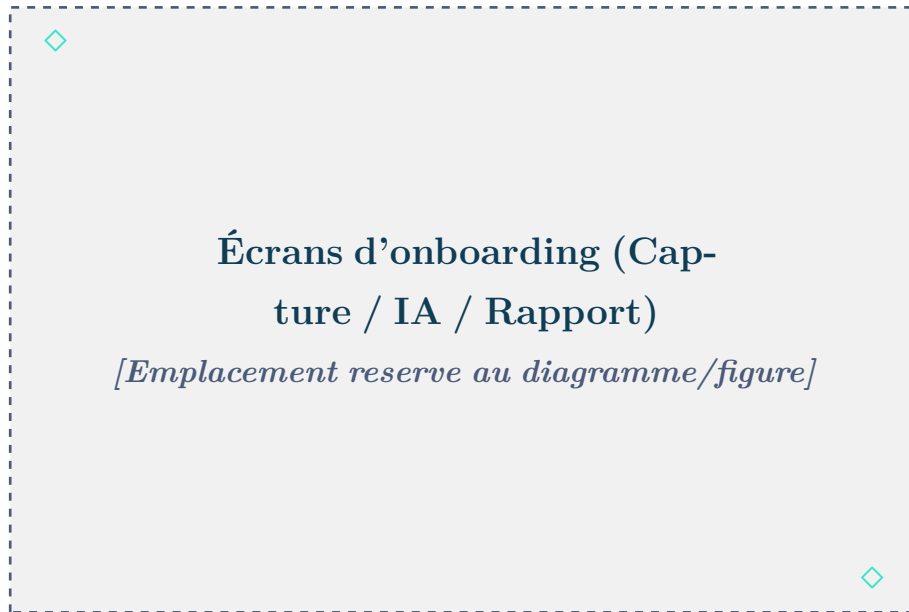


FIGURE 4.1 – Écrans d'onboarding

4.2 Landing Page & Connexion

La page d'accueil met en scène l'identité de PulmoScan AI à travers une section *hero* et un défilement animé qui conduit naturellement l'utilisateur vers le formulaire de connexion. Ce dernier propose la saisie de l'e-mail et du mot de passe, avec vérification locale via SHA-256 et ouverture de session.



FIGURE 4.2 – Landing page et connexion

4.3 Inscription & Vérification

L'écran d'inscription recueille le nom, l'e-mail et le mot de passe du médecin, avec validation des champs. Il est suivi, le cas échéant, d'un écran de vérification par code à six chiffres.

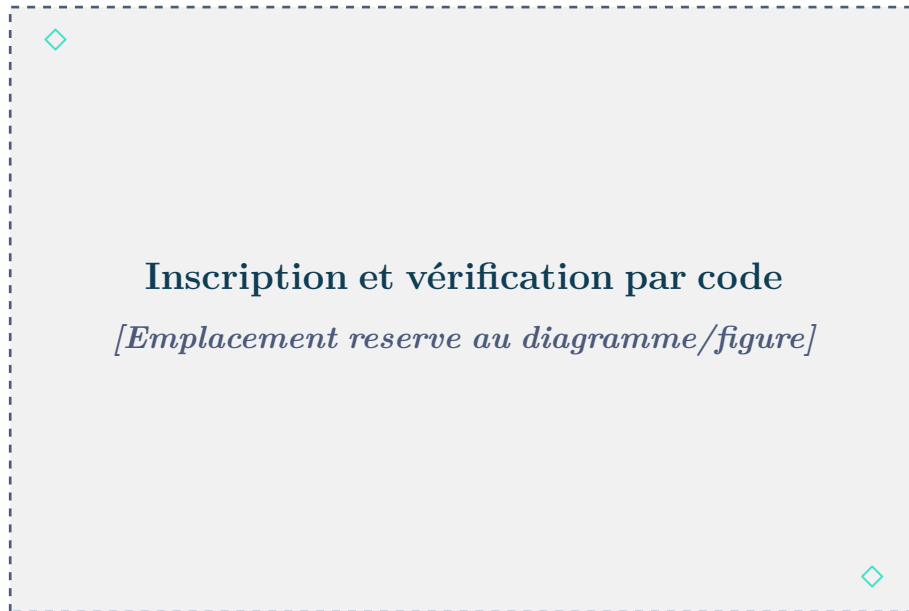


FIGURE 4.3 – Inscription et vérification

4.4 Dashboard & Statistiques

Le tableau de bord offre une vue d'ensemble de l'activité. Des cartes statistiques présentent le nombre de patients, le nombre d'examens et la répartition par sévérité. Une liste des examens récents permet un accès rapide, et des fiches d'information détaillent les deux modèles d'IA embarqués.



FIGURE 4.4 – Dashboard et statistiques

4.5 Liste des Patients

L’écran de liste des patients affiche l’ensemble des dossiers sous forme de cartes, avec une barre de recherche et des badges de statut. Le tableau 4.1 présente les patients de démonstration intégrés à l’application.

TABLE 4.1 – Patients de démonstration					
Nom	Âge	Genre	Antécédents	Diagnostic	
Ahmed Benali	58	Homme	Tabagisme chronique (30 PA)	Emphysema	87% (modéré)
Fatima Zahra	45	Femme	Asthme	Pneumonia	92% (sévère)
Youssef Alaoui	63	Homme	BPCO, hypertension	Effusion	78% (faible)



FIGURE 4.5 – Liste des patients

4.6 Détails Patient & Historique

L'écran de détail d'un patient présente l'intégralité de son dossier — informations administratives, antécédents — ainsi que l'historique chronologique de ses examens, chacun étant cliquable pour accéder aux résultats correspondants.



FIGURE 4.6 – Détails patient et historique

4.7 Création d’Examen

L’écran de création d’examen guide le médecin à travers un assistant en plusieurs étapes : choix du patient, acquisition de l’image radiographique (galerie ou appareil photo), puis déclenchement de l’analyse par IA.

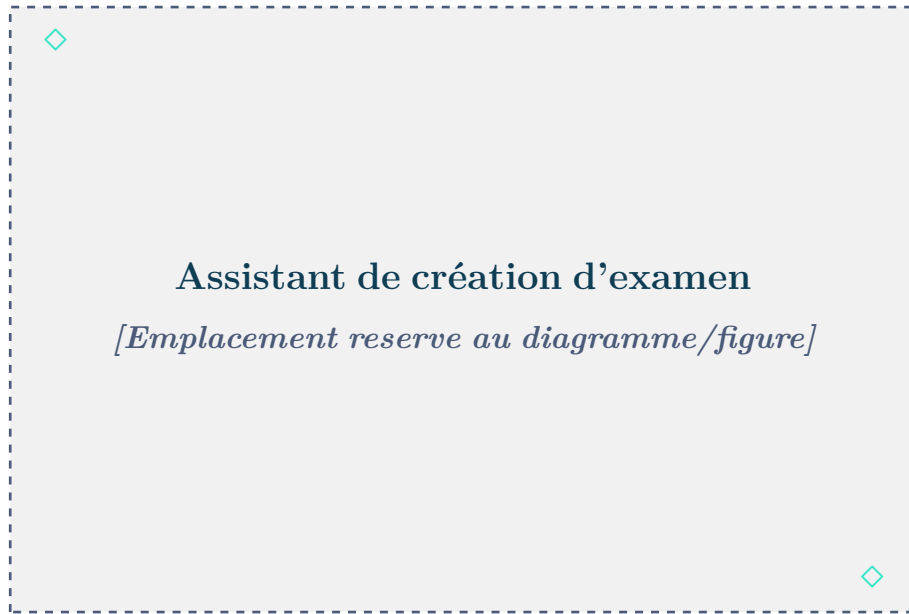


FIGURE 4.7 – Création d’examen

4.8 Résultats de l’Analyse IA

L’écran de résultats est l’élément le plus riche de l’application. Il présente le diagnostic principal avec son indice de confiance et son niveau de sévérité (codé par couleur), un histogramme des scores des 14 pathologies, et la superposition du masque de segmentation U-Net sur l’image. Le médecin peut y ajouter des notes cliniques et générer le rapport PDF.

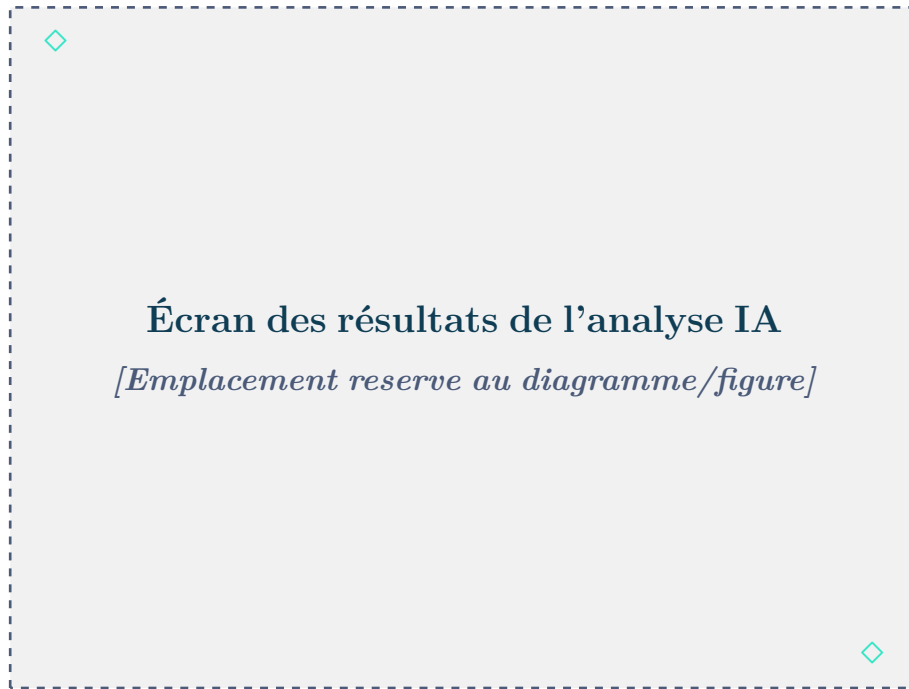


FIGURE 4.8 – Résultats de l'analyse IA

4.9 Profil Médecin

L'écran de profil regroupe les informations du médecin, les réglages de l'application et la possibilité de modifier le mot de passe. Il assure également la déconnexion et la gestion de la session.

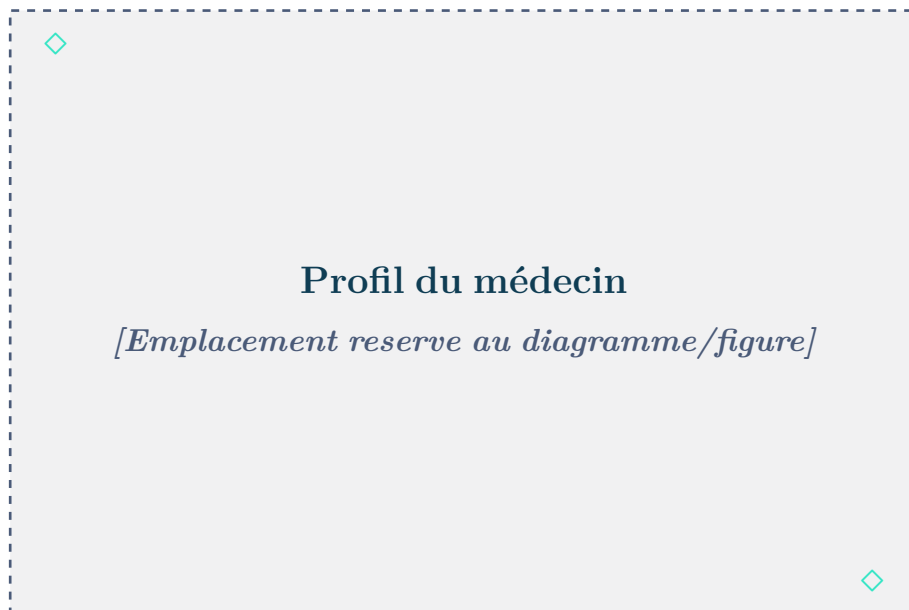


FIGURE 4.9 – Profil médecin

Conclusion

Ce chapitre a parcouru les principales interfaces de PulmoScan AI. Le design system V4 confère à l'application une identité visuelle cohérente et une ergonomie soignée, au service de la clarté du parcours médical. Le chapitre suivant présente les tests menés pour valider la fiabilité de l'ensemble.

Chapitre 5

Tests et Assurance Qualité

Introduction

La fiabilité d'un outil d'aide au diagnostic médical est primordiale. Ce dernier chapitre décrit les démarches de test et d'assurance qualité menées sur PulmoScan AI. Nous abordons successivement les tests du modèle d'IA, les tests fonctionnels, les tests d'interface et d'ergonomie, puis les principales difficultés rencontrées et les solutions apportées.

5.1 Tests du Modèle IA

5.1.1 Validation de la Normalisation (Python)

La première campagne de tests, menée en Python sur les modèles avant leur intégration, visait à valider la stratégie de normalisation. Deux hypothèses ont été comparées : la normalisation par division par 255 (intervalle $[0, 1]$) et la normalisation TorchXRayVision (intervalle $[-1024, 1024]$). Les résultats, résumés dans le tableau 5.1, sont sans appel : la division par 255 produit des scores distribués entre 0,00 et 0,17, exploitables, tandis que la normalisation TorchXRayVision sature l'ensemble des sorties à 1,0 ou 0,0. Ce résultat confirme que la normalisation TorchXRayVision est *déjà intégrée* au modèle exporté, et qu'il ne faut donc pas l'appliquer une seconde fois.

TABLE 5.1 – Validation de la normalisation (tests Python)

Normalisation	Comportement des sorties	Verdict
div255 $[0, 1]$	Scores distribués 0,00–0,17, non saturés.	Valide
TXV $[-1024, 1024]$	Toutes les sorties saturées à 1,0 / 0,0.	Invalide

5.1.2 Métriques de Performance par Pathologie

Les performances du modèle DenseNet121 sont mesurées par l'aire sous la courbe ROC (AUC) pour chacune des 14 pathologies. Le tableau 5.2 présente ces valeurs. Les quatre classes marquées d'un astérisque, dont l'AUC est inférieure à 0,80, sont exclues

du diagnostic principal en raison de leur fiabilité insuffisante, tout en demeurant visibles dans l'histogramme.

TABLE 5.2 – AUC par pathologie (* = exclue du diagnostic principal)

Pathologie	AUC	Pathologie	AUC
Atelectasis	0,818	Pneumonia*	0,768
Consolidation*	0,793	Pleural_Thickening	0,806
Infiltration*	0,703	Cardiomegaly	0,891
Pneumothorax	0,872	Nodule*	0,780
Edema	0,884	Mass	0,863
Emphysema	0,937	Hernia	0,938
Fibrosis	0,883	Effusion	0,882

5.1.3 Comportement sur Images Synthétiques

Des tests complémentaires ont été conduits sur des images synthétiques et sur des radiographies réelles du jeu NIH. Ils ont confirmé que la classe Infiltration ressort systématiquement comme la plus élevée (scores de 0,10 à 0,17), conséquence du déséquilibre des classes dans le jeu d'entraînement — ce qui justifie son exclusion du diagnostic principal. À l'inverse, les classes faibles (Emphysema, Cardiomegaly, Effusion) sont correctement prédites sur les images réelles, validant le bon fonctionnement du pipeline une fois la normalisation et l'ordre des étiquettes corrigés.

5.2 Tests Fonctionnels

Les tests fonctionnels ont vérifié, scénario par scénario, le bon comportement de l'application. Le tableau 5.3 en présente une synthèse.

TABLE 5.3 – Synthèse des tests fonctionnels

Scénario	Résultat attendu	Statut
Inscription d'un médecin	Compte créé, mot de passe haché.	Réussi
Connexion (identifiants valides)	Session ouverte, accès au dashboard.	Réussi
Connexion (identifiants invalides)	Message d'erreur, accès refusé.	Réussi
Création d'un patient	Patient enregistré, CIN unique vérifiée.	Réussi
Création d'un examen avec image	Examen au statut en_attente.	Réussi
Analyse IA	Diagnostic, confiance et sévérité produits.	Réussi
Segmentation U-Net	Masque superposé affiché.	Réussi
Génération de rapport PDF	PDF généré et partageable.	Réussi
Historique des examens	Liste chronologique correcte.	Réussi
Déconnexion	Session effacée.	Réussi

5.3 Tests d'Interface et Ergonomie

Les tests d'interface ont porté sur la cohérence visuelle (respect du design system V4), la lisibilité, la réactivité de la navigation et l'adaptation aux différentes tailles d'écran. Une attention particulière a été portée à la gestion de la barre de navigation gestuelle d'Android, source d'un débordement initialement constaté sur l'onboarding. L'ensemble des écrans a été vérifié sur appareil physique afin de garantir une expérience fluide et sans débordement de mise en page.

5.4 Difficultés Rencontrées et Solutions

Le développement de PulmoScan AI a comporté plusieurs difficultés techniques notables, dont la résolution a été déterminante pour la qualité finale. Le tableau 5.4 les récapitule.

TABLE 5.4 – Difficultés rencontrées et solutions apportées

Difficulté	Solution
Ordre des étiquettes TorchXRayVision $\neq$ ordre alphabétique NIH, entraînant des prédictions erronées.	Correction de l'ordre des étiquettes selon la table TorchXRayVision officielle.
Classes ambiguës à faible AUC (Infiltration, Pneumonia, Nodule, Consolidation) dominant le diagnostic.	Exclusion de ces classes du diagnostic principal, tout en les conservant dans l'histogramme.
Normalisation TXV $[-1024, 1024]$ saturant toutes les sorties.	Retour à la normalisation par division par 255, validée par test Python.
Débordement (overflow) lié à la barre de navigation gestuelle Android sur l'onboarding.	Prise en compte de <code>MediaQuery.of(context).padding.bottom</code> .
Incohérence des libellés de sévérité en base ('Modérée' vs 'modere').	Normalisation vers les jetons du thème V4.
Condition de course lors de la migration de version de la base.	Implémentation correcte du callback <code>onUpgrade</code> .

Conclusion

Les campagnes de tests ont confirmé la fiabilité du pipeline d'inférence et le bon fonctionnement des fonctionnalités de l'application. La résolution méthodique des difficultés rencontrées — notamment l'ordre des étiquettes et la stratégie de normalisation — a permis d'atteindre des prédictions exploitables sur les classes fiables. PulmoScan AI répond ainsi aux exigences fonctionnelles et non fonctionnelles fixées au début du projet.

Conclusion Générale

Le projet PulmoScan AI nous a permis de concevoir et de réaliser une application mobile complète de pré-diagnostic des pathologies pulmonaires, fonctionnant intégralement hors ligne. À travers ce travail, nous avons relevé un double défi : maîtriser un framework de développement multiplateforme moderne (Flutter) et intégrer, sur les contraintes d'un appareil mobile, deux modèles d'intelligence artificielle — un réseau DenseNet121 pour la classification de 14 pathologies et un réseau U-Net pour la segmentation pulmonaire.

L'architecture offline-first retenue répond directement à la problématique initiale : elle garantit la confidentialité des données médicales, leur disponibilité permanente et une indépendance totale vis-à-vis du réseau. La mise au point du pipeline d'inférence a constitué la partie la plus exigeante du projet ; elle nous a appris l'importance cruciale de la compréhension fine des modèles utilisés, notamment de leur normalisation interne et de l'ordre exact de leurs sorties.

Sur le plan des compétences, ce projet a renforcé notre maîtrise du développement mobile, de la modélisation UML, de la conception de bases de données relationnelles, de la sécurité applicative et de l'apprentissage profond appliqué à l'imagerie médicale. La répartition du travail — Kenza Foudali sur les modèles d'IA (notebooks Colab, conversion TFLite), la conception de la base SQLite et l'UI/UX, Adnane El Hajji sur l'architecture Flutter, l'intégration, le backend optionnel, les tests et le déploiement — nous a également formés au travail collaboratif.

Plusieurs **perspectives** d'évolution se dégagent. À court terme, l'activation effective du backend de synchronisation permettrait le partage sécurisé des dossiers entre praticiens. À moyen terme, le ré-entraînement ou le *fine-tuning* des modèles sur des données locales améliorerait la fiabilité sur les classes actuellement exclues. Enfin, l'ajout d'une fonction d'explicabilité (cartes de chaleur de type Grad-CAM) renforcerait la confiance des praticiens dans les prédictions de l'application.

PulmoScan AI démontre qu'il est possible de mettre une assistance au diagnostic par intelligence artificielle, performante et respectueuse de la vie privée, à la portée de la première ligne de soins, y compris dans des contextes à connectivité limitée. Nous espérons que ce travail contribuera, modestement, à la réflexion sur une santé numérique accessible et éthique.

Webographie / Références

1. Huang, G., Liu, Z., van der Maaten, L., & Weinberger, K. Q. (2017). *Densely Connected Convolutional Networks*. IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2017).
2. Ronneberger, O., Fischer, P., & Brox, T. (2015). *U-Net : Convolutional Networks for Biomedical Image Segmentation*. MICCAI 2015.
3. Wang, X., Peng, Y., Lu, L., Lu, Z., Bagheri, M., & Summers, R. M. (2017). *ChestX-ray8 : Hospital-scale Chest X-ray Database and Benchmarks on Weakly-Supervised Classification and Localization of Common Thorax Diseases*. CVPR 2017.
4. Cohen, J. P., et al. (2022). *TorchXRayVision : A library of chest X-ray datasets and models*. github.com/mlmed/torchxrayvision
5. Flutter Team (2024). *Flutter Documentation*. flutter.dev
6. Google (2024). *TensorFlow Lite — ML for Mobile and Edge Devices*. tensorflow.org/lite
7. National Institutes of Health (2017). *NIH Chest X-ray Dataset*. nihcc.app.box.com
8. Ministère de la Santé du Maroc (2022). *Rapport sur les maladies respiratoires*. sante.gov.ma